ROOMWISE

Smart Campus Room Finder and Scheduling System

OOP using Java and DBMS Project Report

Report submitted by

Adithya Narayana Anand (250701022)

Aadithya T (250701001)

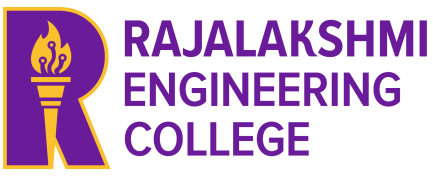
Mitran Reddy (250701422)

Bachelor of Engineering in Computer Science and Engineering

Semester III | 2026

Rajalakshmi Engineering College, Thandalam

An Autonomous Institution Affiliated to Anna University



RAJALAKSHMI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this is the bonafide record of the project titled **"ROOMWISE"** done by **Adithya Narayana Anand (250701022)**, **Aadithya T (250701001)** and **Mitran Reddy (250701422)**, who carried out the project work under our supervision during the academic year **2026**.

Dr K Ananthajothi
Faculty in-charge
OOP using Java
Dept of CSE

Ms S Kaviya
Faculty in-charge
DBMS
Dept of CSE

Submitted for the Practical Examination held on _____

Internal Examiner

External Examiner

STUDENT DECLARATION

We, **Adithya Narayana Anand (250701022)**, **Aadithya T (250701001)** and **Mitran Reddy (250701422)**, students of Semester III, Department of Computer Science and Engineering, Rajalakshmi Engineering College, hereby declare that the project report entitled **"RoomWise - Smart Campus Room Finder and Scheduling System"** records the project submitted for Object-Oriented Programming using Java and Database Management Systems.

The implementation uses Java, a terminal menu, JDBC and MySQL. This report describes the source code and database schema supplied with the project, including the implemented OOP concepts, database operations, verification results and remaining limitations. External technical material and the supplied timetable data are identified in the References section.

We accept responsibility for reviewing the report, understanding the implementation and explaining the project during the practical demonstration and evaluation. Each team member must be able to trace a user action through the service layer to its SQL operation and explain the associated object-oriented concepts.

Place: Chennai
Date: _____

Adithya Narayana Anand (250701022)
Signature: _____

Aadithya T (250701001)
Signature: _____

Mitran Reddy (250701422)
Signature: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to the management and faculty of Rajalakshmi Engineering College, Thandalam, for providing the opportunity and facilities to undertake this project.

We thank the Department of Computer Science and Engineering for the academic support provided during the development of RoomWise. The project helped us connect classroom concepts in object-oriented programming and relational database management with an application that addresses campus room availability.

We extend our thanks to **Dr K Ananthajothi**, Faculty in-charge of OOP using Java, for guidance in the study and application of object-oriented programming concepts. We also thank **Ms S Kaviya**, Faculty in-charge of DBMS, for guidance in database design, SQL and Java database connectivity.

We acknowledge the supplied theory and laboratory timetable datasets used to exercise the application. We also acknowledge the official Java and MySQL documentation consulted while implementing the JDBC connection, SQL constraints and transactions.

We thank our classmates and everyone who contributed suggestions and feedback. The team is responsible for verifying the final submission and preparing for its demonstration.

Adithya Narayana Anand

Aadithya T

Mitran Reddy

ABSTRACT

RoomWise is a Java terminal application that helps users discover free campus rooms and maintain room schedules. A recurring weekly timetable and dated reservations are stored in MySQL and accessed through hand-written JDBC. The supplied dataset contains 157 rooms and 4,167 timetable records, providing a realistic basis for availability searches and database demonstrations.

The application supports Create, Read, Update and Delete operations for rooms, bookings and timetable entries. Users can search availability at a particular time or for an entire interval, inspect occupied events, create reservations, reschedule them and cancel them. A room with dependent schedules cannot be deleted until those dependencies are removed.

The Java implementation demonstrates encapsulation through validated private fields, abstraction through an abstract event class and a persistence interface, inheritance through booking and timetable subclasses, and runtime polymorphism through date matching and event descriptions. Overloaded search methods demonstrate compile-time polymorphism. Separate model, DAO, service, UI, utility and exception packages make the responsibilities visible.

Database integrity is supported by primary keys, foreign keys, unique room names, time-range checks and explicit transactions. A singleton InnoDB guard row serializes scheduling writes so room and teacher conflicts are checked before a write commits. Nineteen automated tests passed, including CRUD, constraint enforcement, rollback, safe initialization and concurrent booking attempts. This report includes implementation diagrams, the exact SQL schema, source-code images, application screenshots and a team demonstration plan.

Keywords: Java; OOP; terminal; JDBC; MySQL; CRUD; room scheduling; transactions; concurrency.

Implementation snapshot: verified on 9 November 2026. The assignment notice stated a deadline of 05/11/26 (5 November 2026); this report does not assert that submission occurred by that date.

TABLE OF CONTENTS

BONAFIDE CERTIFICATE **2**

STUDENT DECLARATION **3**

ACKNOWLEDGEMENT **4**

ABSTRACT **5**

TABLE OF CONTENTS **6**

LIST OF FIGURES **9**

LIST OF TABLES **10**

CHAPTER 1: INTRODUCTION **11**

1.1 Background and problem statement 11

1.2 Project objectives 11

1.3 Existing approach, proposed solution and scope 12

1.4 Users and boundaries 12

1.5 Organization of this report 12

CHAPTER 2: REQUIREMENTS AND FEASIBILITY **13**

2.1 Functional requirements 13

2.2 Non-functional requirements 13

2.3 Technical requirements and feasibility 14

2.4 Feasibility study 14

2.5 Acceptance criteria 14

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN **15**

3.1 Layered architecture 15

3.2 Use cases and interaction design 16

3.3 Domain class design 17

3.4 Design decisions 17

3.5 Booking flow and concurrency design 18

3.6 Time interval model 18

CHAPTER 4: DATABASE DESIGN 19

4.1 Database overview and relationships 19

4.2 Room and booking data dictionary 20

4.3 Booking constraints and indexes 20

4.4 Timetable and synchronization data dictionary 21

4.5 Integrity rules 21

4.6 Functional dependencies and normalization 22

4.7 Dataset provenance 22

4.8 CRUD, SQL and transaction semantics 23

4.9 ACID in this implementation 23

CHAPTER 5: IMPLEMENTATION 24

5.1 Project structure and responsibility mapping 24

5.2 OOP concept-to-code map 24

5.3 Encapsulation and validation 25

5.4 Abstraction and shared scheduling behavior 26

5.5 Inheritance and method overriding 27

5.6 Runtime and compile-time polymorphism 28

5.7 Interfaces and package integration 29

5.8 JDBC connectivity and resource management 30

5.9 Transaction implementation and exception handling 31

5.10 CRUD implementation and generated keys 32

5.11 Object mapping, UI integration and setup 33

5.12 Build and run procedure 33

5.13 Simplified terminal implementation 34

5.14 Step-by-step terminal input and search . . . 35

Simple algorithm used for availability . . . 35

Terminal command guide . . . 35

CHAPTER 6: VERIFICATION AND TEST PLAN 36

6.1 Verification method and environment . . . 36

6.2 Build and startup checks . . . 36

6.3 Automated test cases . . . 37

6.4 Integrity, concurrency and migration tests . . . 38

6.5 Manual verification and evidence limits . . . 38

6.6 Demonstration and team preparation . . . 39

6.7 Team readiness . . . 39

CHAPTER 7: SECURITY, LIMITATIONS, AND MAINTENANCE 40

7.1 Implemented security and integrity controls . . . 40

7.2 Known limitations . . . 40

7.3 Performance and maintenance plan . . . 41

7.4 Troubleshooting . . . 41

CHAPTER 8: CONCLUSION AND FUTURE WORK 42

8.1 Conclusion . . . 42

8.2 Prioritized future work . . . 42

REFERENCES 43

APPENDIX A: SQL SCHEMA 44

Appendix A / SQL schema continued 45

A.1 Initial data and migration notes . . . 45

A.2 Useful read-only demonstration queries . . . 45

APPENDIX B: OUTPUT SCREENSHOTS 46

LIST OF FIGURES

Figure 3.1. Implemented UI-service-DAO-database architecture. 15
Figure 3.2. Use-case view of browsing and schedule maintenance. 16
Figure 3.3. Actual domain inheritance and polymorphism. 17
Figure 3.4. Atomic schedule mutation and error path. 18
Figure 4.1. Entity-relationship diagram of the implemented four-table schema. 19
Figure 5.1. Encapsulated Room state and validated setter. 25
Figure 5.2. Abstract event model, common state and overlap rule. 26
Figure 5.3. Booking inherits the common event contract. 27
Figure 5.4. TimetableSlot overrides recurrence and description behavior. 27
Figure 5.5. Simple event loop and overloaded search entry point. 28
Figure 5.6. CampusDao persistence interface for all domain CRUD operations. 29
Figure 5.7. External configuration and JDBC connection creation. 30
Figure 5.8. Guarded write transaction with commit and rollback. 31
Figure 5.9. Booking and timetable INSERT, UPDATE and DELETE methods. 32
Figure 5.10. Mapping JDBC ResultSet rows into domain objects. 33
Figure 5.11. Terminal control flow and recoverable input errors. 34
Figure 5.12. Reading booking fields before constructing a validated model. 35
Figure B.1. Terminal menu. 47
Figure B.2. Free-room search output. 48
Figure B.3. Create and update a room. 49
Figure B.4. Create and read a booking. 50
Figure B.5. Update and read a booking. 51
Figure B.6. Reject an overlapping booking. 52
Figure B.7. Polymorphic occupied-event output. 53
Figure B.8. Timetable create, update and read. 54
Figure B.9. Delete records and verify cleanup. 55

LIST OF TABLES

Table 1.1. Project objectives and implemented outcomes. 11
Table 1.2. Existing approach and proposed system. 12
Table 2.1. Functional requirements traceability. 13
Table 2.2. Software and demonstration environment. 14
Table 3.1. Principal use cases and results. 16
Table 4.1. Room and booking columns. 20
Table 4.2. Timetable and guard columns. 21
Table 4.3. Database design trade-offs. 22
Table 4.4. Implemented CRUD coverage. 23
Table 5.1. Source package map. 24
Table 5.2. OOP concepts and implementation evidence. 24
Table 5.3. Terminal menu guide. 35
Table 6.1. Observed automated test result. 36
Table 6.2. Model and functional integration tests. 37
Table 6.3. Integrity, concurrency and migration tests. 38
Table 6.4. Terminal demonstration coverage and limits. 38
Table 6.5. Suggested demonstration sequence. 39
Table 7.1. Security and scheduling limitations. 40
Table 7.2. Maintenance responsibilities. 41
Table 8.1. Future development priorities. 42
Table B.1. Terminal evidence provenance. 46

CHAPTER 1: INTRODUCTION

1.1 Background and problem statement

Campus teaching rooms are shared by multiple departments, sections and teachers. A room may be free for one period yet required by another class shortly afterward. Checking an isolated timetable row is therefore insufficient: a useful room finder must consider the requested date, weekly classes, existing reservations and the whole requested interval.

RoomWise consolidates timetable and booking information into one application. The problem addressed is: given a date and a point or interval in time, identify rooms that have no conflicting event, and maintain the records that determine that result. A second requirement is to prevent a teacher or room from being booked for overlapping events through the application.

1.2 Project objectives

Objective	Implemented outcome
Find available rooms	Point-in-time search and whole-interval search; next scheduled start shown.
Keep schedules current	CRUD for rooms, dated bookings and recurring timetable entries.
Demonstrate core Java OOP	Validated models, abstract superclass, subclasses, overrides, overloads, interface, packages and custom exceptions.
Integrate Java with DBMS	Plain JDBC, parameter binding, object mapping, generated keys, constraints and transactions.
Provide reproducible evidence	19 automated tests, source paths, diagrams and recorded terminal output images.

Table 1.1. Project objectives and implemented outcomes.

1.3 Existing approach, proposed solution and scope

The supplied timetable exports provide class details but do not themselves provide an integrated room-search and reservation workflow. Manually consulting these exports can overlook a dated booking or an overlapping interval. The proposed solution adds a structured relational store and a Java application that presents both recurring and dated events through a common model.

Aspect	Manual / separate timetable lookup	RoomWise
Availability	User compares multiple entries.	Service combines timetable events and dated bookings.
Record maintenance	Direct file or SQL edits.	Validated console CRUD commands.
Conflict handling	Depends on manual checking.	Atomic room and teacher overlap checks on writes.
Evidence	Static timetable content.	Persistent database state and reproducible tests.

Table 1.2. Existing approach and proposed system.

1.4 Users and boundaries

Campus users browse free rooms and occupied events. A trusted local operator maintains rooms and schedules. These are usage responsibilities, not authenticated roles: the current application has no login screen or authorization system. Anyone able to launch it with valid database credentials can reach its management functions.

The scope is a same-day scheduling prototype with a weekly recurring timetable. It does not manage semesters, public holidays, room equipment, capacities, attendance, payment, notifications or university-wide identity. Dated bookings can be entered for any date; restricting past dates is not an implemented policy.

1.5 Organization of this report

The following chapters cover requirements and feasibility; analysis and design; database design; implementation; verification; security and maintenance; and conclusions. Appendix A contains the exact schema. Appendix B provides application output evidence. Source-code images appear beside the relevant implementation explanation.

CHAPTER 2: REQUIREMENTS AND FEASIBILITY

2.1 Functional requirements

ID	Requirement	Implementation / verification
FR-01	Connect to a configured local MySQL database.	DatabaseConnection; startup smoke check.
FR-02	Read available rooms at a date and time.	CampusService.search(date, time).
FR-03	Find rooms free for an entire interval.	search(date, from, until); boundary tests.
FR-04	Show occupied timetable and booking events.	occupied(...); runtime subclass dispatch.
FR-05	Create, read, update and delete rooms.	Manage rooms submenu.
FR-06	Create, read, update and delete bookings.	Manage bookings submenu; generated keys.
FR-07	Create, read, update and delete timetable rows.	Manage timetable submenu; conflict checks.
FR-08	Reject room or teacher overlaps.	JDBC conflict queries while holding guard row.
FR-09	Preserve existing data at startup.	Idempotent schema setup and explicit empty-only seed.
FR-10	Report invalid input without silently saving it.	Validated models, custom exceptions and UI messages.

Table 2.1. Functional requirements traceability.

2.2 Non-functional requirements

Correctness takes priority over visual decoration. Inputs must satisfy schema limits; SQL values must be parameterized; writes must commit or roll back as a unit; and existing records must survive application restart. The user interface must identify errors and offer a repeatable workflow. Code must remain understandable enough for all three team members to explain.

CHAPTER 2 / CONTINUED

2.3 Technical requirements and feasibility

Component	Project requirement / verified configuration
Language and runtime	Java 17 compatibility; local compilation and execution with JDK 21.
Build	Maven; JUnit Jupiter 5.11.4; Surefire 3.5.2.
Database	MySQL 8.0.16 or newer for enforced CHECK constraints; InnoDB tables.
JDBC driver	MySQL Connector/J 8.3.0, declared in pom.xml.
User interface	Java Scanner/PrintStream terminal menu; no desktop toolkit.
Host	Windows used for verification. Java code is portable; launcher is a Windows .bat file.
Suggested demonstration machine	Modern dual-core CPU, 4 GB RAM and sufficient space for JDK, Maven and MySQL. This is a practical recommendation, not a measured minimum.

Table 2.2. Software and demonstration environment.

2.4 Feasibility study

Technical feasibility: Java provides desktop widgets, date/time classes, collections and JDBC. MySQL provides relational constraints and transactions. The implemented dataset is small enough for a classroom demonstration, although the current service scans in-memory event lists instead of issuing optimized availability queries.

Operational feasibility: the numbered menu provides search, rooms, bookings and timetable operations. An operator needs to understand date/time formats and use Manage rooms / Read to identify a valid room ID. No internet connection is required after dependencies and the local database are installed.

Economic and schedule feasibility: the project uses a local Java/MySQL toolchain and needs no paid hosting. The main effort is implementation, data validation and testing. Completion evidence in this report is dated 9 November 2026; the stated assignment deadline remains 5 November 2026.

2.5 Acceptance criteria

The prototype is accepted for this implementation snapshot when all three domain entities support CRUD, invalid intervals and foreign keys are rejected, both OOP dispatch mechanisms are visible in source, JDBC data persists between connections, concurrent conflicting writes produce one winner, and the test suite and demonstration can be repeated.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 Layered architecture

Main is the composition root. It initializes the schema, creates JdbcCampusDao, injects it into CampusService and launches ConsoleMenu directly. The UI imports the service and model packages; the service depends on CampusDao. SQL is kept in the DAO package and connection setup in the utility package.

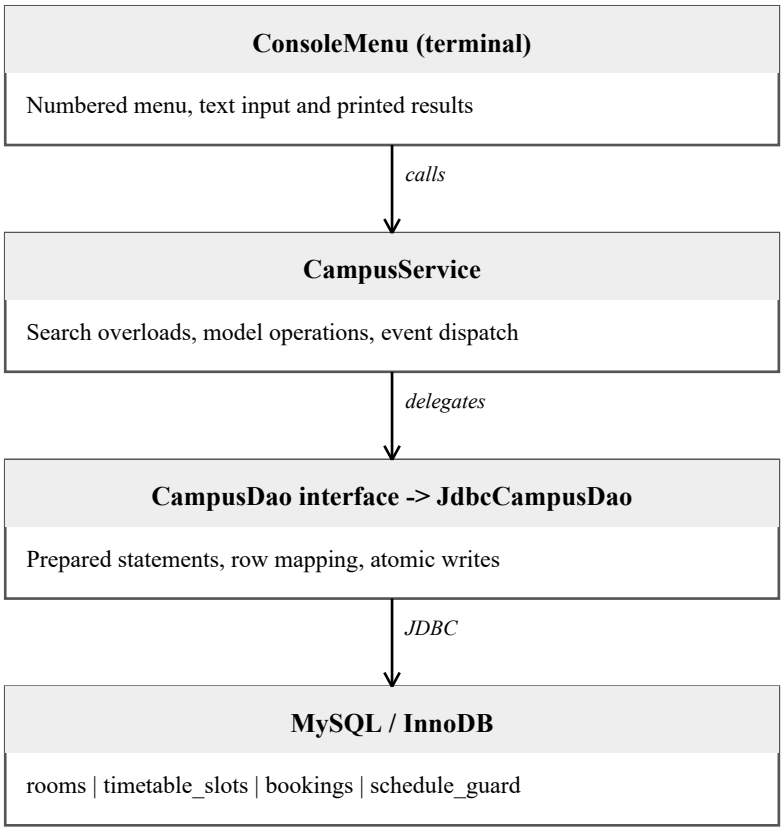
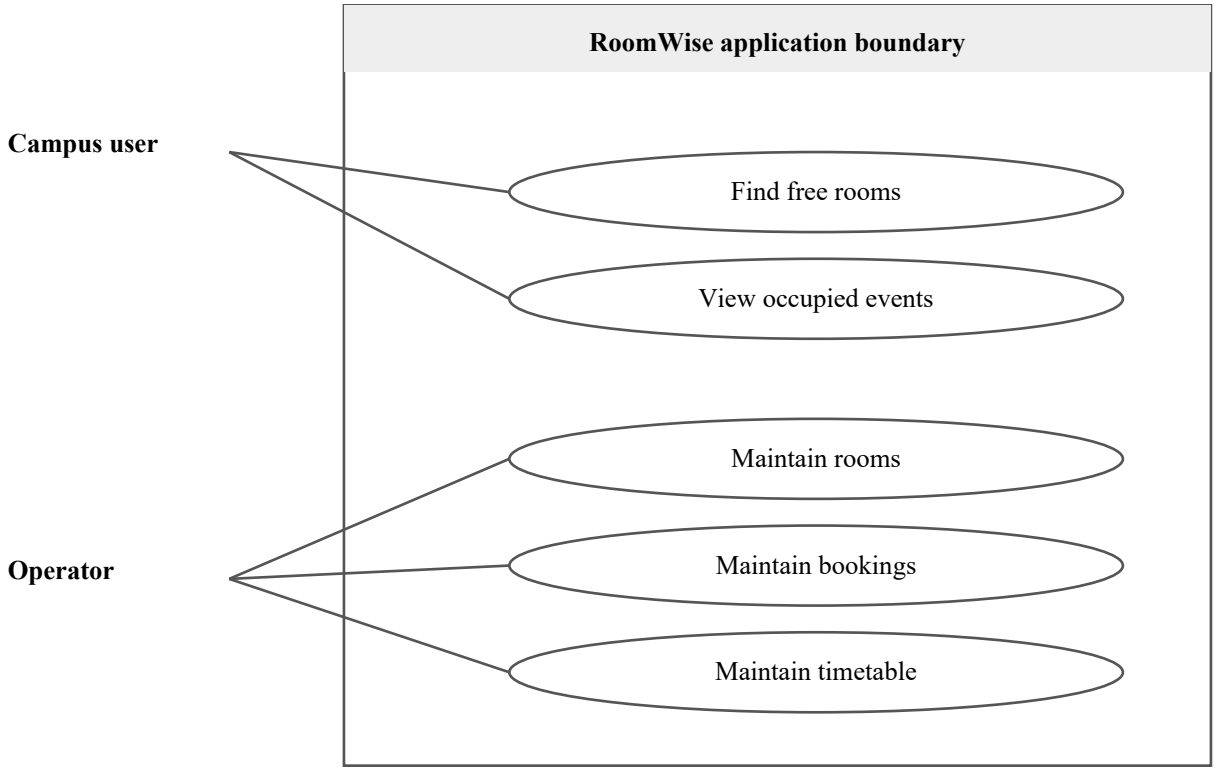


Figure 3.1. Implemented UI-service-DAO-database architecture.

Constructor injection makes the dependency direction explicit. The persistence interface hides JDBC mechanics from the service, while returned Room, Booking and TimetableSlot objects carry validated domain state. DatabaseException preserves a cause internally while the UI displays a concise message.

3.2 Use cases and interaction design



Actors describe usage responsibilities; authentication and role enforcement are not implemented.

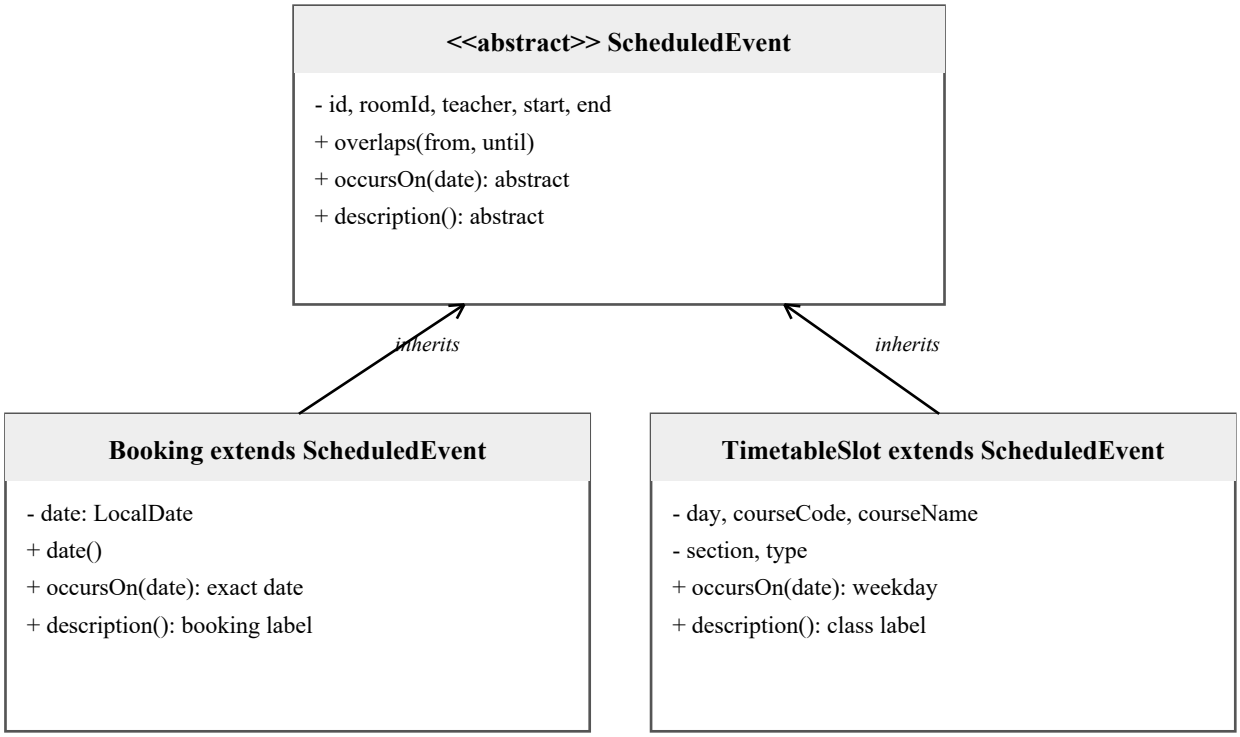
Figure 3.2. Use-case view of browsing and schedule maintenance.

Use case	Precondition	Main result
Search availability	Date/time parse successfully.	List of free rooms or active events.
Create reservation	Room exists; model is valid; no overlapping room/teacher event.	Generated ID returned after commit.
Edit reservation	Selected booking exists.	New values persist; its old row is excluded from conflict checks.
Delete room	No timetable or booking depends on the room.	One row deleted; otherwise an explanation is shown.

Table 3.1. Principal use cases and results.

CHAPTER 3 / CONTINUED

3.3 Domain class design



CampusService uses List<ScheduledEvent> for both concrete subclasses.

Figure 3.3. Actual domain inheritance and polymorphism.

ScheduledEvent centralizes identity, room reference, teacher and validated start/end times. Booking adds an exact date. TimetableSlot adds a recurring weekday and course information. Their different occursOn implementations let the service select the events for a date without duplicating a complete search algorithm for each subclass.

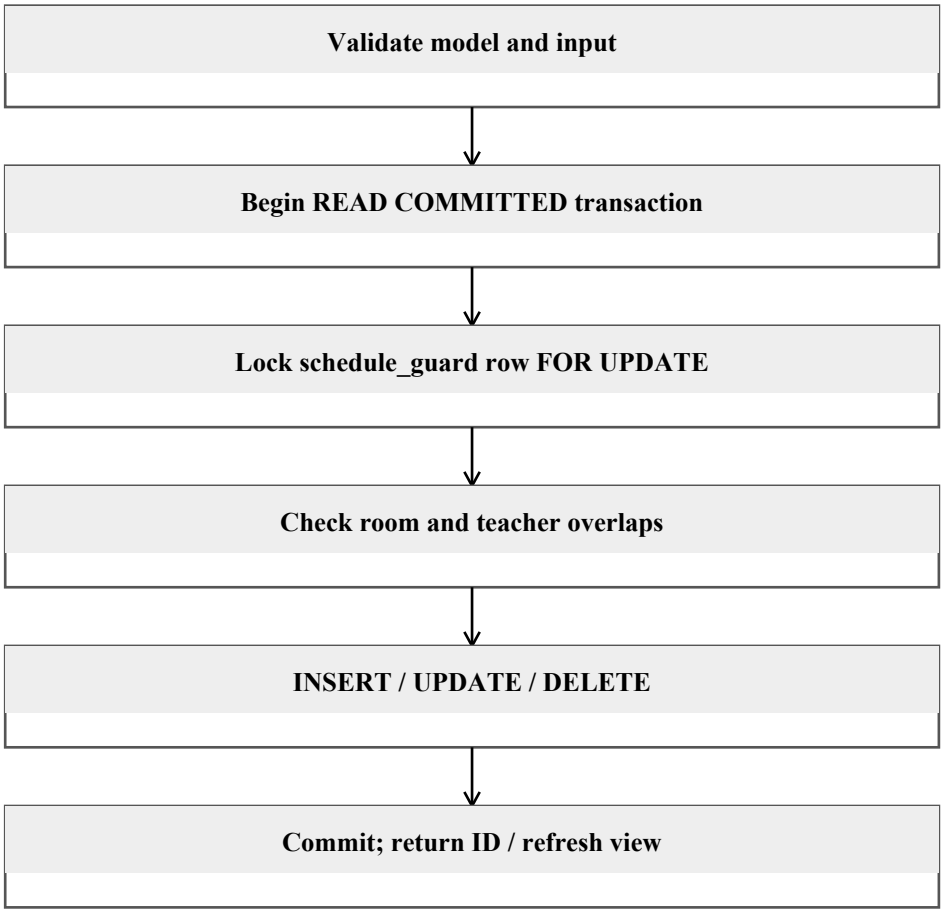
Room encapsulates its primary key and name; the setter validates nonblank text and the database column limit. FreeRoom is an immutable result record, not a persisted table. The hierarchy models the two scheduling concepts already present in the actual schema, rather than adding unused subclasses only for the report.

3.4 Design decisions

The domain objects do not execute SQL. CampusDao supplies a stable persistence contract. JdbcCampusDao maps rows to those objects and implements the transaction boundary. The UI is responsible for terminal prompts and printed results. This separation makes the link between an OOP concept and a database action easy to demonstrate.

CHAPTER 3 / CONTINUED

3.5 Booking flow and concurrency design



Any validation, conflict or SQL failure -> rollback; show an actionable error.

Figure 3.4. Atomic schedule mutation and error path.

Every DAO write takes the same guard-row lock. This conservative design serializes room, booking and timetable mutations, including teacher conflicts across different rooms. After the first transaction commits, a waiting transaction checks the committed state at READ COMMITTED isolation and rejects an overlap. The lock is released by commit or rollback.

The design trades write throughput for clarity and correctness in a small campus prototype. A future implementation could use more selective locks. Direct SQL issued by another tool does not automatically acquire the guard; such external writes must follow the same protocol or be restricted.

3.6 Time interval model

```
overlap = existingStart < requestedEnd
       && existingEnd   > requestedStart
```

Intervals are half-open: the start is included and the end excluded. A 09:00-10:00 event may be followed by a 10:00-11:00 event. Equal start and end values, reversed intervals and intervals crossing midnight are rejected. Recurring timetable entries are matched by weekday; bookings are matched by exact date.

CHAPTER 4: DATABASE DESIGN

4.1 Database overview and relationships

The implemented database has three domain tables and one internal synchronization table. Room names are stored once in rooms and referenced by both schedule tables. A room can have zero or many timetable rows and zero or many dated bookings. Every schedule row must reference one existing room.

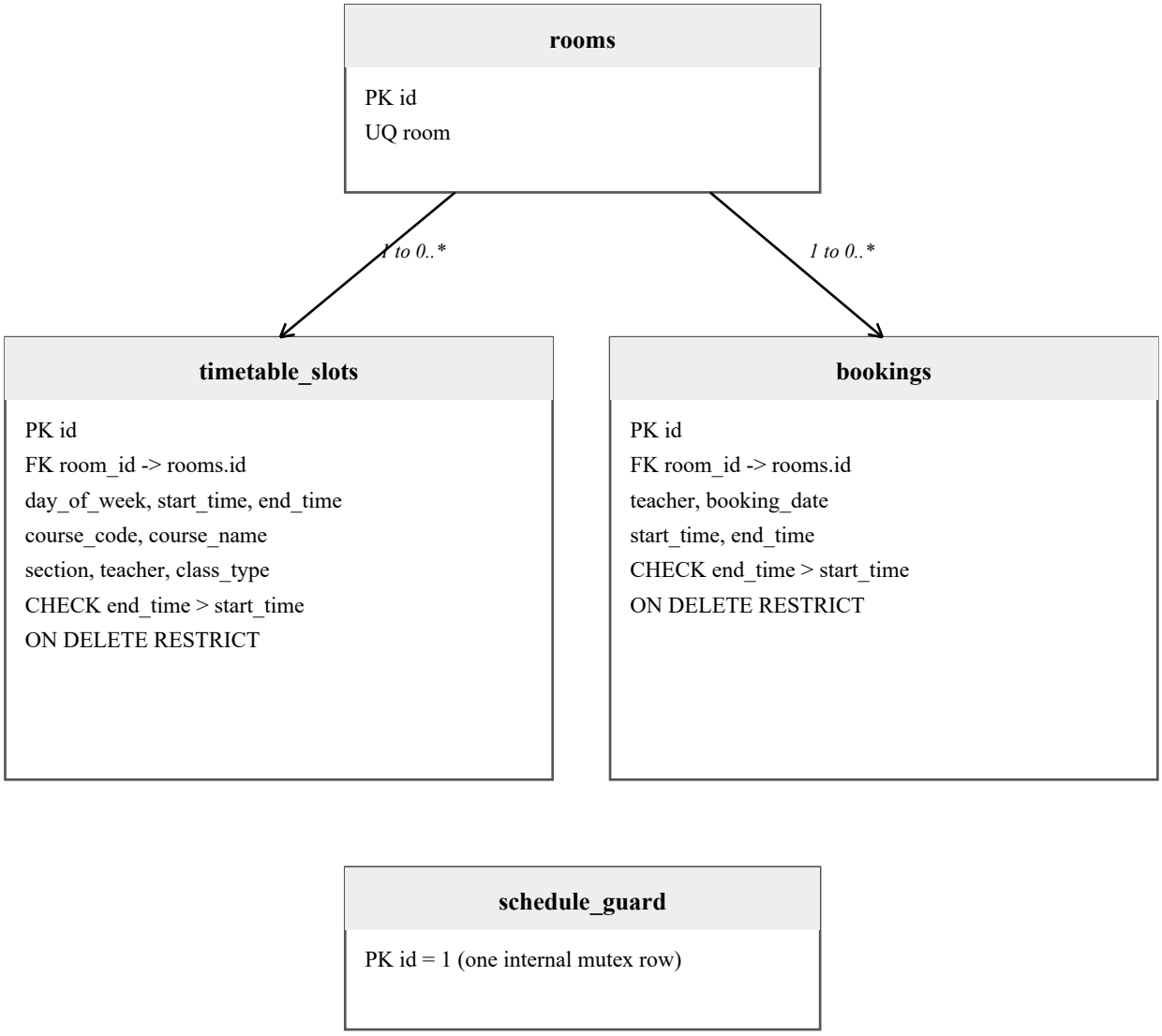


Figure 4.1. Entity-relationship diagram of the implemented four-table schema.

schedule_guard is deliberately independent of the domain relationships. Its single row is a transaction mutex, not a user, booking or course entity. The application creates it idempotently and locks it during writes.

4.2 Room and booking data dictionary

Table / column	Type	Constraint / meaning
rooms.id	INT	Primary key; auto-increment.
rooms.room	VARCHAR(40)	Unique, NOT NULL room label.
bookings.id	INT	Primary key; auto-increment.
bookings.teacher	VARCHAR(150)	NOT NULL; teacher display name.
bookings.room_id	INT	NOT NULL foreign key to rooms.id.
bookings.booking_date	DATE	Exact date of the reservation.
bookings.start_time	TIME	Inclusive start; NOT NULL.
bookings.end_time	TIME	Exclusive end; must be greater than start_time.

Table 4.1. Room and booking columns.

4.3 Booking constraints and indexes

The primary key identifies a booking independently of its editable time and teacher fields. The foreign key prevents an orphan room reference. ON DELETE RESTRICT prevents direct SQL from deleting a room that still has bookings. chk_booking_times rejects zero-length and reversed intervals.

idx_bookings_date_room_time begins with booking_date, followed by room_id, start_time and end_time. It supports date/room lookup patterns; the OR branch that also checks normalized teacher text can require additional scanning. No unmeasured claim is made that every conflict query uses only this index.

There is no unique constraint on a whole booking interval because exact equality is not the same as overlap: 09:00-10:00 and 09:30-10:30 have different values but conflict. Overlap prevention is implemented by parameterized checks inside the guarded transaction.

4.4 Timetable and synchronization data dictionary

Column	Type	Meaning / rule
timetable_slots.id	INT	Auto-increment primary key.
room_id	INT	Foreign key to rooms.id; NOT NULL.
day_of_week	ENUM	MONDAY through SUNDAY.
start_time / end_time	TIME	Same-day interval; end greater than start.
course_code	VARCHAR(40)	Source course identifier; not declared unique.
course_name	VARCHAR(180)	Source course title.
section	VARCHAR(180)	Department/semester/group description.
teacher	VARCHAR(150)	Source teacher name.
class_type	ENUM	THEORY or LAB.
schedule_guard.id	TINYINT	Primary key with CHECK id=1; one initialized mutex row.

Table 4.2. Timetable and guard columns.

The timetable foreign key uses ON DELETE RESTRICT. Startup detects the legacy fk_slot_room cascade relationship and replaces it with fk_slot_room_restrict without deleting timetable rows. The migration has a dedicated integration test. idx_slots_day_room_time indexes weekday, room and time bounds.

4.5 Integrity rules

The database enforces identity, required values, supported enum domains, room-name uniqueness, valid room references and positive time ranges. The Java model also validates names, lengths and time values before JDBC is called. Room and teacher overlap checks are application business rules evaluated within a database transaction; they are not SQL CHECK constraints or triggers.

4.6 Functional dependencies and normalization

Within rooms, id determines room and the unique room name also identifies its row. In bookings, id determines teacher, room_id, booking_date, start_time and end_time. In timetable_slots, id determines the remaining class attributes. These are the keys actually declared in the schema.

The tables store one value per column and have single-column surrogate primary keys, so there are no partial dependencies on a composite primary key. However, the timetable retains repeated course names, section descriptions and teacher names from the source exports. The report therefore does not claim a fully normalized course, faculty and offering model or BCNF certification.

A course code is not assumed to uniquely determine a course title across all supplied source records. Earlier source inspection identified code/name collisions. Introducing a course table requires a verified identity rule, and teacher normalization requires stable staff identifiers. The current schema preserves the source information instead of silently merging records.

Design	Benefit	Trade-off
Separate rooms table	Avoids repeating the room label in every schedule row.	Requires a join or Java mapping for display.
Dated bookings separate from weekly classes	Keeps recurrence and exact-date semantics explicit.	Availability must combine both collections.
Course/teacher text retained	Preserves original export details.	Repeated text and name-based teacher identity remain.
Singleton guard	Simple atomic validation across all writers.	Serializes writes and must be honored by external tools.

Table 4.3. Database design trade-offs.

4.7 Dataset provenance

database/data/theory.json and labs.json are the retained source exports. database/real_data.sql contains 157 room rows and 4,167 timetable rows. The original conversion uses 24-hour afternoon times. The new --seed operation loads this dataset only when all domain tables are empty; it never overwrites an existing timetable. Imported source overlaps are retained for fidelity and are not represented as newly accepted reservations.

4.8 CRUD, SQL and transaction semantics

Entity	Create	Read	Update	Delete
Room	INSERT rooms(room)	SELECT id,room	UPDATE room WHERE id	DELETE unused row
Booking	INSERT; return key	SELECT bookings	UPDATE fields WHERE id	DELETE booking row
Timetable	INSERT; return key	SELECT timetable_slots	UPDATE fields WHERE id	DELETE timetable row

Table 4.4. Implemented CRUD coverage.

INSERT returns the generated key through Statement.RETURN_GENERATED_KEYS and ResultSet. UPDATE and DELETE verify that exactly one row was affected; a missing selection produces NotFoundException. Editing a booking or timetable entry excludes its own primary key from conflict queries, so an unchanged row does not conflict with itself.

```
SELECT id FROM schedule_guard WHERE id = 1 FOR UPDATE;
-- overlap query uses bound parameters
WHERE day_of_week = ?
  AND (room_id = ? OR LOWER(TRIM(teacher)) = LOWER(TRIM(?)))
  AND start_time < ? AND end_time > ? AND id <> ?
```

For a new booking, the DAO checks weekly classes on its weekday and dated bookings on its exact date. For a recurring class, it checks other classes on the weekday and bookings whose date has the same weekday. Teacher matching trims surrounding spaces and compares without case; it is not a substitute for a unique staff ID.

4.9 ACID in this implementation

Atomicity: a write commits as one unit or rolls back on SQL/runtime failure. **Consistency:** model checks, business conflict checks and relational constraints reject invalid states. **Isolation:** READ COMMITTED plus the shared guard row serializes application writers. **Durability:** committed rows are managed by InnoDB; operational durability still depends on MySQL configuration, storage and backups.

CHAPTER 5: IMPLEMENTATION

5.1 Project structure and responsibility mapping

Package / file	Responsibility
roomwise.Main	Initializes database; wires DAO to service; starts the console, or explicitly seeds with --seed.
model	Room, ScheduledEvent, Booking, TimetableSlot and FreeRoom.
dao	CampusDao contract; JdbcCampusDao CRUD and transactions; DatabaseInitializer schema/seed.
service.CampusService	Availability, overloads, occupied-event selection and CRUD facade.
ui	ConsoleMenu; no DAO imports.
util	DatabaseConnection and Validation.
exception	DatabaseException, BookingConflictException and NotFoundException.
src/test/java/roomwise	ModelTest and DatabaseIntegrationTest.

Table 5.1. Source package map.

5.2 OOP concept-to-code map

Concept	Actual implementation
Encapsulation	Room private fields and validated setName; event private final fields.
Abstraction	abstract ScheduledEvent; CampusDao interface.
Inheritance	Booking and TimetableSlot extend ScheduledEvent.
Runtime polymorphism	occursOn and description overrides invoked through ScheduledEvent.
Compile-time polymorphism	Three search overloads; two occupied overloads.
Packages	model / dao / service / ui / util / exception.
Exception handling	Custom runtime types, rollback, try-with-resources and UI error messages.

Table 5.2. OOP concepts and implementation evidence.

CHAPTER 5 / CONTINUED

5.3 Encapsulation and validation

model/Room.java

```
1  package roomwise.model;
2  import roomwise.util.Validation;
3  /** Encapsulates room identity and a validated name. */
4  public final class Room {
5      private final int id;
6      private String name;
7      /** Zero denotes an unsaved room; database IDs are positive. */
8      public Room(int id, String name) {
9          if (id < 0) throw new IllegalArgumentException("Room ID cannot be negative.");
10         this.id = id; setName(name);
11     }
12     /** Returns the database key. */
13     public int id() { return id; }
14     /** Returns the room name. */
15     public String name() { return name; }
16     /** Enforces the NOT NULL / VARCHAR(40) domain before a database write. */
17     public void setName(String name) { this.name = Validation.text(name, 40, "Room name"); }
18     @Override public String toString() { return id + " - " + name; }
19 }
```

Figure 5.1. Encapsulated Room state and validated setter.

Source: `src/main/java/roomwise/model/Room.java`, lines 1-19. This code image is rendered directly from the submitted source; long lines wrap at the continuation marker.

Room.id is private and final, so a caller cannot rewrite its persistent identity. Room.name is private and can change only through setName. Validation.text trims the value, rejects blank input and enforces the VARCHAR(40) limit. A failed setter leaves the previous valid name unchanged; ModelTest verifies this behavior.

The event superclass uses private final fields because an event update is represented by a newly constructed object with the same ID. This ensures that every proposed time interval is validated before the DAO receives it. Encapsulation here protects business invariants, not merely field visibility.

CHAPTER 5 / CONTINUED

5.4 Abstraction and shared scheduling behavior

model/ScheduledEvent.java

```
1  package roomwise.model;
2  import java.time.*;
3  import roomwise.util.Validation;
4  /** Abstraction shared by a dated Booking and a recurring TimetableSlot. */
5  public abstract class ScheduledEvent {
6      private final int id, roomId;
7      private final String teacher;
8      private final LocalTime start, end;
9      /** Establishes invariant identity, teacher and same-day time fields. */
10     protected ScheduledEvent(int id, int roomId, String teacher, LocalTime start, LocalTime
 > end) {
11         if (id < 0 || roomId <= 0) throw new IllegalArgumentException("Invalid record or
 > room ID.");
12         Validation.times(start, end);
13         this.id=id; this.roomId=roomId; this.teacher=Validation.text(teacher,150,"Teacher");
14         this.start=start; this.end=end;
15     }
16     /** Returns the persistent identity, or zero for a new event. */
17     public final int id() { return id; }
18     /** Returns the rooms foreign key. */
19     public final int roomId() { return roomId; }
20     /** Returns the teacher's display name. */
21     public final String teacher() { return teacher; }
22     /** Returns the inclusive interval start. */
23     public final LocalTime start() { return start; }
24     /** Returns the exclusive interval end. */
25     public final LocalTime end() { return end; }
26     /** Uses half-open intervals, so adjacent events do not collide. */
27     public final boolean overlaps(LocalTime from, LocalTime until) {
28         return start.isBefore(until) && end.isAfter(from);
29     }
30     /** Subclasses implement dated versus recurring occurrence rules. */
31     public abstract boolean occursOn(LocalDate date);
32     /** Runtime polymorphism supplies the display label of each event. */
33     public abstract String description();
34 }
```

Figure 5.2. Abstract event model, common state and overlap rule.

Source: `src/main/java/roomwise/model/ScheduledEvent.java`, lines 1-34. This code image is rendered directly from the submitted source; long lines wrap at the continuation marker.

An abstract class cannot be instantiated directly. ScheduledEvent defines what all scheduled items have in common and leaves two variable behaviors to its subclasses. Its constructor validates identity, teacher text and interval length. The final accessors and overlaps method provide one common interpretation of interval boundaries.

CHAPTER 5 / CONTINUED

5.5 Inheritance and method overriding

model/Booking.java

```
1  package roomwise.model;
2  import java.time.*;
3  import java.util.Objects;
4  /** A one-date reservation inherits common scheduling state and validation. */
5  public final class Booking extends ScheduledEvent {
6      private final LocalDate date;
7      /** Constructs a validated booking. */
8      public Booking(int id, int roomId, String teacher, LocalDate date, LocalTime start,
>  LocalTime end) {
9          super(id,roomId,teacher,start,end); this.date=Objects.requireNonNull(date,"Booking
>  date is required");
10     }
11     /** Returns the exact reservation date. */
12     public LocalDate date() { return date; }
13     @Override public boolean occursOn(LocalDate day) { return date.equals(day); }
14     @Override public String description() { return "Booking | " + teacher(); }
15 }
```

Figure 5.3. Booking inherits the common event contract.

Source: `src/main/java/roomwise/model/Booking.java`, lines 1-15. This code image is rendered directly from the submitted source; long lines wrap at the continuation marker.

model/TimetableSlot.java

```
21     /** Returns the source course code. */ public String courseCode() { return courseCode; }
22     /** Returns the course name. */ public String courseName() { return courseName; }
23     /** Returns the class group. */ public String section() { return section; }
24     /** Returns THEORY or LAB. */ public Type type() { return type; }
25     @Override public boolean occursOn(LocalDate date) { return date.getDayOfWeek()==day; }
26     @Override public String description() { return type + " | " + courseCode + " " +
>  courseName + " | " + teacher(); }
27 }
```

Figure 5.4. TimetableSlot overrides recurrence and description behavior.

Source: `src/main/java/roomwise/model/TimetableSlot.java`, lines 21-27. This code image is rendered directly from the submitted source; long lines wrap at the continuation marker.

Booking matches one LocalDate; TimetableSlot matches the date's DayOfWeek. Both objects are valid ScheduledEvent instances. This is a meaningful is-a relationship: both can occupy a room, overlap a time interval and provide a display description, but they differ in recurrence. The report does not claim that unrelated GUI inheritance supplies the only domain OOP evidence.

CHAPTER 5 / CONTINUED

5.6 Runtime and compile-time polymorphism

service/CampusService.java

```
32      private List<ScheduledEvent> events(LocalDate date) {
33          List<ScheduledEvent> all = new ArrayList<>(dao.slots());
34          all.addAll(dao.bookings());
35          List<ScheduledEvent> matching = new ArrayList<>();
36          for (ScheduledEvent event : all) {
37              // Runtime polymorphism selects the subclass date rule.
38              if (event.occursOn(date)) matching.add(event);
39          }
40          return matching;
41      }
42
43      /** Overload one: search at the current minute on the given date. */
44      public List<FreeRoom> search(LocalDate date) {
45          return search(date, LocalTime.now().withSecond(0).withNano(0));
46      }
47
```

Figure 5.5. Simple event loop and overloaded search entry point.

Source: `src/main/java/roomwise/service/CampusService.java`, lines 32-47. This code image is rendered directly from the submitted source; long lines wrap at the continuation marker.

events(date) combines timetable and booking objects into List<ScheduledEvent>. Calling event.occursOn(date) dispatches to the appropriate override at runtime. The occupied display similarly calls description through the superclass reference. Adding a compatible event type would not require duplicating the whole presentation loop.

search(date), search(date, at) and search(date, from, until) use the same method name with different parameter lists. Java chooses the overload at compile time. The interval overload tests the entire requested period; the point overload reports the next occupied start after the selected time. These are separate use cases, not interchangeable results.

CHAPTER 5 / CONTINUED

5.7 Interfaces and package integration

dao/CampusDao.java

```
1  package roomwise.dao;
2  import java.util.List;
3  import roomwise.model.*;
4  /** Persistence abstraction: service/UI code does not depend on JDBC details. */
5  public interface CampusDao {
6      /** Reads all rooms. */ List<Room> rooms();
7      /** Reads all recurring timetable rows. */ List<TimetableSlot> slots();
8      /** Reads all dated bookings. */ List<Booking> bookings();
9      /** Creates a room and returns its generated primary key. */ int createRoom(Room room);
10     /** Updates a named room, rejecting missing IDs. */ void updateRoom(Room room);
11     /** Deletes an unused room; dependent schedules prevent deletion. */ void deleteRoom(int
 >  id);
12     /** Creates a conflict-checked booking. */ int createBooking(Booking booking);
13     /** Updates a booking atomically while excluding its old row from checks. */ void
 > updateBooking(Booking booking);
14     /** Deletes a booking by primary key. */ void deleteBooking(int id);
15     /** Creates a conflict-checked weekly class. */ int createSlot(TimetableSlot slot);
16     /** Updates a weekly class with room and teacher checks. */ void
 > updateSlot(TimetableSlot slot);
17     /** Deletes a weekly class. */ void deleteSlot(int id);
18 }
```

Figure 5.6. CampusDao persistence interface for all domain CRUD operations.

Source: `src/main/java/roomwise/dao/CampusDao.java`, lines 1-18. This code image is rendered directly from the submitted source; long lines wrap at the continuation marker.

CampusService receives a CampusDao through its constructor. Main supplies a JdbcCampusDao instance. This is interface-based abstraction: the service knows the available operations without owning a JDBC connection or SQL string. The current implementation uses one JDBC implementation; the interface provides a boundary for testing and future persistence changes.

A complete booking path is: terminal input -> new Booking -> CampusService.createBooking -> CampusDao.createBooking -> JdbcCampusDao transaction -> MySQL INSERT -> generated ID -> refreshed UI. The Booking object links inheritance and validation to a concrete database operation.

CHAPTER 5 / CONTINUED

5.8 JDBC connectivity and resource management

util/DatabaseConnection.java

```
1  package roomwise.util;
2  import java.io.*;
3  import java.nio.file.*;
4  import java.sql.*;
5  import java.util.Properties;
6  import roomwise.exception.DatabaseException;
7  /** Plain JDBC connection factory. Credentials remain outside the built JAR. */
8  public final class DatabaseConnection {
9      private DatabaseConnection() {}
10     /** Opens a fresh connection using the local file or an explicit config path. */
11     public static Connection open() {
12         Path path = Path.of(System.getProperty("roomwise.config", "db.properties"));
13         try (InputStream input = Files.newInputStream(path)) {
14             Properties p = new Properties(); p.load(input);
15             String url = System.getProperty("roomwise.db.url", p.getProperty("db.url"));
16             if (url == null || p.getProperty("db.user") == null)
17                 throw new IOException("Required connection settings are absent");
18             url += (url.contains("?") ? "&" : "?") +
 > "connectTimeout=10000&socketTimeout=30000";
19             return DriverManager.getConnection(url, p.getProperty("db.user"),
 > p.getProperty("db.password"));
20         } catch (IOException | SQLException e) {
21             throw new DatabaseException("Cannot connect to MySQL. Check the server and local
 >  db.properties.", e);
22         }
23     }
24 }
```

Figure 5.7. External configuration and JDBC connection creation.

Source: `src/main/java/roomwise/util/DatabaseConnection.java`, lines 1-24. This code image is rendered directly from the submitted source; long lines wrap at the continuation marker.

DatabaseConnection opens db.properties from the working directory, or from the path supplied by the roomwise.config system property. The build no longer embeds that credential file in the JAR. The roomwise.db.url override is used for isolated tests and demonstrations. The report and code images do not contain the local username, password or complete private connection settings.

The factory uses DriverManager and the declared Connector/J driver. Connections have finite connect and socket timeouts. Try-with-resources closes the configuration stream, connections, statements and result sets. IO and SQL errors are wrapped in DatabaseException with a user-facing message. Prepared parameters keep user values separate from SQL syntax. [2]

CHAPTER 5 / CONTINUED

5.9 Transaction implementation and exception handling

dao/JdbcCampusDao.java

```
15      private <T> T write(Work<T> work) {
16          try (Connection c=DatabaseConnection.open()) {
17              c.setTransactionIsolation(Connection.TRANSACTION_READ_COMMITTED);
18              c.setAutoCommit(false);
19              try {
20                  // One InnoDB guard row serializes all application schedule mutations.
21                  try (PreparedStatement lock=c.prepareStatement("SELECT id FROM
 > schedule_guard WHERE id=1 FOR UPDATE");
22                       ResultSet rows=lock.executeQuery()) {
23                      if (!rows.next()) throw new SQLException("Scheduling guard is missing");
24                  }
25                  T value=work.run(c); c.commit(); return value;
26              } catch (SQLException | RuntimeException e) {
27                  try { c.rollback(); } catch (SQLException rollback) {
 > e.addSuppressed(rollback); }
28                  throw e;
29              }
30          } catch (SQLException e) { throw failure(e); }
31      }
32      private DatabaseException failure(SQLException e) {
33          String message=e.getSQLState()!=null && e.getSQLState().startsWith("23")
34                  ? "Database constraint rejected the change. Check duplicate names, room
 > references and time values."
35                  : "Database operation failed. Check MySQL and the application
 > configuration.";
36          return new DatabaseException(message,e);
37      }
```

Figure 5.8. Guarded write transaction with commit and rollback.

Source: `src/main/java/roomwise/dao/JdbcCampusDao.java`, lines 15-37. This code image is rendered directly from the submitted source; long lines wrap at the continuation marker.

The shared write helper sets the isolation level, disables auto-commit and locks schedule_guard. The callback then performs the entity-specific work. A successful callback commits. A SQLException or RuntimeException triggers rollback; a rollback failure is attached as a suppressed exception so the original failure is not lost. [3]

BookingConflictException expresses a business collision; NotFoundException identifies a stale/missing record. DatabaseException wraps JDBC failures. The console catches recoverable operation errors, prints a clear message and returns to its menu. End of input exits cleanly. This is explicit error reporting rather than swallowing an exception.

CHAPTER 5 / CONTINUED

5.10 CRUD implementation and generated keys

dao/JdbcCampusDao.java

```
119     }
120     @Override public int createBooking(Booking b) {
121         return write(c -> { checkConflict(c,b); return execute(c,
122                 "INSERT INTO bookings(teacher,room_id,booking_date,start_time,end_time)
  > VALUES (?,?,?,?,?)",
123                 b.teacher(),b.roomId(),b.date(),b.start(),b.end()); });
124     }
125     @Override public void updateBooking(Booking b) {
126         write(c -> { exists(c,"bookings",b.id()); checkConflict(c,b); change(c,
127                 "UPDATE bookings SET
  > teacher=?,room_id=?,booking_date=?,start_time=?,end_time=? WHERE id=?",
128                 b.teacher(),b.roomId(),b.date(),b.start(),b.end(),b.id()); return null; });
129     }
130     @Override public void deleteBooking(int id) {
131         write(c -> { change(c,"DELETE FROM bookings WHERE id=?",id); return null; });
132     }
133     @Override public int createSlot(TimetableSlot t) {
134         return write(c -> { checkConflict(c,t); return execute(c,
135                 "INSERT INTO timetable_slots(room_id,day_of_week,start_time,end_time,course_
  > code,course_name,section,teacher,class_type) VALUES (?,?,?,?,?,?,?,?,?)",
136                 t.roomId(),t.day().name(),t.start(),t.end(),t.courseCode(),t.courseName(),t.
  > section(),t.teacher(),t.type().name()); });
137     }
138     @Override public void updateSlot(TimetableSlot t) {
139         write(c -> { exists(c,"timetable_slots",t.id()); checkConflict(c,t); change(c,
140                 "UPDATE timetable_slots SET room_id=?,day_of_week=?,start_time=?,end_time=?,
  > course_code=?,course_name=?,section=?,teacher=?,class_type=? WHERE id=?",
141                 t.roomId(),t.day().name(),t.start(),t.end(),t.courseCode(),t.courseName(),t.
  > section(),t.teacher(),t.type().name(),t.id()); return null; });
142     }
143     @Override public void deleteSlot(int id) {
144         write(c -> { change(c,"DELETE FROM timetable_slots WHERE id=?",id); return null; });
145     }
```

Figure 5.9. Booking and timetable INSERT, UPDATE and DELETE methods.

Source: `src/main/java/roomwise/dao/JdbcCampusDao.java`, lines 119-145. This code image is rendered directly from the submitted source; long lines wrap at the continuation marker.

The booking and timetable writes bind values taken from validated model objects. Create returns a database-generated ID. Update keeps the primary key and replaces editable values only after conflict checks. Delete removes the selected row inside the same guarded transaction. Room deletion checks dependents and is also protected by foreign keys.

CHAPTER 5 / CONTINUED

5.11 Object mapping, UI integration and setup

dao/JdbcCampusDao.java

```
46      @Override public List<Booking> bookings() {
47          return read(c -> {
48              List<Booking> list=new ArrayList<>();
49              try (PreparedStatement s=c.prepareStatement("SELECT * FROM bookings ORDER BY
 > booking_date,start_time,id"); ResultSet r=s.executeQuery()) {
50                  while(r.next()) list.add(new
 > Booking(r.getInt("id"),r.getInt("room_id"),r.getString("teacher"),
51                          r.getObject("booking_date",LocalDate.class),r.getObject("start_time"
 > ,LocalTime.class),r.getObject("end_time",LocalTime.class)));
52              } return list;
53          });
54      }
55      @Override public List<TimetableSlot> slots() {
56          return read(c -> {
57              List<TimetableSlot> list=new ArrayList<>();
58              try (PreparedStatement s=c.prepareStatement("SELECT * FROM timetable_slots ORDER
 >  BY day_of_week,start_time,id"); ResultSet r=s.executeQuery()) {
59                  while(r.next()) list.add(new
 > TimetableSlot(r.getInt("id"),r.getInt("room_id"),r.getString("teacher"),
60                          DayOfWeek.valueOf(r.getString("day_of_week")),r.getObject("start_tim
 > e",LocalTime.class),r.getObject("end_time",LocalTime.class),
61                          r.getString("course_code"),r.getString("course_name"),r.getString("s
 > ection"),TimetableSlot.Type.valueOf(r.getString("class_type"))));
62              } return list;
63          });
64      }
```

Figure 5.10. Mapping JDBC ResultSet rows into domain objects.

Source: `src/main/java/roomwise/dao/JdbcCampusDao.java`, lines 46-64. This code image is rendered directly from the submitted source; long lines wrap at the continuation marker.

Read operations reconstruct Room, Booking and TimetableSlot objects using typed LocalDate/LocalTime values. Those constructors apply the same domain invariants used by new UI input. The service can therefore work with objects, while persistence details remain in the DAO.

5.12 Build and run procedure

```
cd RoomWise
mvn package dependency:copy-dependencies -DincludeScope=runtime
java -cp "target/classes;target/dependency/*" roomwise.Main
java -cp "target/classes;target/dependency/*" roomwise.Main --seed
```

Create the MySQL database and configure a local db.properties before running. For a new empty schema only, run Main --seed after startup creates the tables. Existing data is preserved and nonempty seeds are refused. The Windows run_roomwise.bat launcher starts the console directly and cleans generated output after the application exits. On Unix-like systems the classpath separator is a colon.

CHAPTER 5 / CONTINUED

5.13 Simplified terminal implementation

The application uses one terminal menu instead of a graphical interface. Main creates the DAO, service and ConsoleMenu. There are no Swing classes, event listeners, windows or UI threads. The application package depends on java.base and java.sql, with Connector/J supplying database connectivity.

ConsoleMenu now separates each task into a small named method: printMenu, showFreeRooms, showOccupied, rooms, bookings and timetable. readBooking and readSlot collect one field at a time, then construct the model. This replaces dense one-line expressions with a sequence that can be followed during a classroom explanation.

ui/ConsoleMenu.java

```
36      public void start() {
37          while (true) {
38              printMenu();
39              if (!input.hasNextLine()) return;
40              try {
41                  switch (read("Choose: ")) {
42                      case "0": out.println("Goodbye."); return;
43                      case "1": showFreeRooms(); break;
44                      case "2": showOccupied(); break;
45                      case "3": rooms(); break;
46                      case "4": bookings(); break;
47                      case "5": timetable(); break;
48                      case "6": showFreeInterval(); break;
49                      default: out.println("Use a number from 0 to 6.");
50                  }
51              } catch (NoSuchElementException endOfInput) {
52                  out.println("Input ended. Returning to the terminal.");
53                  return;
54              } catch (RuntimeException error) {
55                  out.println("Action failed: " + error.getMessage());
56              }
57          }
58      }
59
60      private void printMenu() {
61          out.println("\n========== ROOMWISE ==========");
62          out.println("1. Find free rooms at a time");
63          out.println("2. View occupied events");
64          out.println("3. Manage rooms");
65          out.println("4. Manage bookings");
```

Figure 5.11. Terminal control flow and recoverable input errors.

Source: `src/main/java/roomwise/ui/ConsoleMenu.java`, lines 36-65. This code image is rendered directly from the submitted source; long lines wrap at the continuation marker.

The constructor also accepts Scanner and PrintStream, allowing the same menu to be exercised with recorded input and captured output. This is used by the terminal integration test and the Appendix B evidence recorder; it does not replace JDBC with mock data.

CHAPTER 5 / CONTINUED

5.14 Step-by-step terminal input and search

ui/ConsoleMenu.java

```
157     private Booking readBooking(int id) {
158         int roomId = readId("Room");
159         String teacher = read("Teacher: ");
160         LocalDate date = readDate();
161         LocalTime start = readTime("Start");
162         LocalTime end = readTime("End");
163         return new Booking(id, roomId, teacher, date, start, end);
164     }
165
```

Figure 5.12. Reading booking fields before constructing a validated model.

Source: `src/main/java/roomwise/ui/ConsoleMenu.java`, lines 157-165. This code image is rendered directly from the submitted source; long lines wrap at the continuation marker.

The model constructor checks the interval and teacher text. The service delegates persistence to CampusDao. The JDBC implementation holds the shared database lock while checking collisions and changing rows. Each layer has a separate job, so removing the graphical interface does not remove the required abstraction, inheritance or DBMS concepts.

Simple algorithm used for availability

```
For each room:
    busy = false
    nextStart = end of day
    For each event on the requested date:
        Skip events belonging to another room
        If the event contains the selected time: busy = true
        Remember the earliest future start
    If not busy: add the room to the result
```

CampusService uses ordinary for loops and boolean variables instead of chained stream filters. The whole-interval search gathers overlapping events and excludes their room IDs. The existing overlap formula and transaction rules are preserved, and the same regression tests pass.

Terminal command guide

Main choice	Purpose
1 / 2	Point-in-time free rooms / active events.
3 / 4 / 5	Room / booking / timetable maintenance; choose C, R, U or D.
6	Find rooms free for the complete start-to-end interval.
0	Exit.
DELETE	Required confirmation text before deletion.

Table 5.3. Terminal menu guide.

CHAPTER 6: VERIFICATION AND TEST PLAN

6.1 Verification method and environment

Verification was performed on 9 November 2026 with JDK 21, Maven and local MySQL. Model tests run without a database. Integration tests create a uniquely named temporary database, apply the real schema, exercise JDBC operations and remove that test database afterward. They never clear the configured campus database.

```
mvn test
mvn test "-Droomwise.integration=true"
```

The first command runs four model tests and skips the explicitly gated integration class. The second runs all nineteen tests and requires database privileges to create and remove the temporary test schema. A skipped integration class must not be reported as a full database test pass.

Suite	Tests	Failures	Errors	Skipped in full run
ModelTest	4	0	0	0
DatabaseIntegrationTest	15	0	0	0
Total	19	0	0	0

Table 6.1. Observed automated test result.

6.2 Build and startup checks

Maven test and package completed successfully. A console startup against the configured campus database reached "Database ready. Existing records preserved.", displayed the new menu and exited normally. The built JAR was inspected and contains no db.properties credential file. The verified local MySQL version was 8.0.40; the campus database retained 157 rooms and 4,167 timetable records after the upgrade.

Fresh terminal sessions were executed through the actual ConsoleMenu, service and JDBC layers in an isolated database. Appendix B contains readable images rendered from their captured output; they are identified as transcript images, not photographs of a terminal window. The original input sequences and full logs are retained with the report source.

6.3 Automated test cases

ID	Test condition	Observed result
UT-01	Invalid/blank room rename and valid rename.	Rejected invalid name; prior value preserved.
UT-02	Half-open overlap and adjacent intervals.	Adjacent slots allowed; actual overlap detected.
UT-03	Equal times, blank teacher, long room name.	Model construction rejected.
UT-04	Superclass references to both subclasses.	Exact-date and weekly dispatch differed correctly.
IT-01	Room CRUD, duplicate name, stale ID.	CRUD passed; duplicate and missing ID rejected.
IT-02	Booking create/read/update/delete.	Generated ID and updated time persisted.
IT-03	Timetable create/read/update/delete.	All four operations passed.
IT-04	Room conflict and same teacher in another room.	Both rejected; adjacent booking accepted.
IT-05	Move a booking into an occupied interval.	Conflict raised; original booking retained.
IT-06	Recurring class overlaps a dated reservation.	New class rejected.
IT-07	Search at boundaries and over full interval.	Both subclasses included; boundaries correct.

Table 6.2. Model and functional integration tests.

6.4 Integrity, concurrency and migration tests

ID	Test condition	Observed result
IT-08	Delete referenced room; invalid FK and time via SQL.	Database rejected all invalid changes.
IT-09	Quoted SQL-like room label as bound value.	Stored as text; room table remained intact.
IT-10	Restart initializer with edited data.	Room and timetable rows preserved.
IT-11	Two simultaneous bookings for the same room.	Exactly one successful commit.
IT-12	Two rooms, same teacher, simultaneous times.	Exactly one successful commit.
IT-13	Explicit initial seed, then repeat on nonempty data.	157 rooms / 4,167 slots loaded; second seed refused.
IT-14	Legacy CASCADE foreign key migration.	Rows retained; direct room delete then rejected.
IT-15	Scripted terminal update/read/delete and invalid ID.	Correct output, data changes and error recovery.

Table 6.3. Integrity, concurrency and migration tests.

6.5 Manual verification and evidence limits

Check	Evidence / status
Terminal search	Point search recorded for 2026-11-09 at 09:00; 91 free rooms.
Room maintenance	Console creates room 158 and renames it to DEMO-SEMINAR.
Booking maintenance	Console creates booking 1, reads it, updates its end to 19:30, then reads the persisted value.
Conflict handling	Overlapping booking rejected through the terminal menu.
Timetable maintenance	Console creates and updates slot 4168; occupied query reads its new 17:00-17:30 interval.
Deletion	Console removes the booking, timetable slot and unused demo room; final counts return to baseline.
Not measured	High-volume load, network outages and long-running stress.

Table 6.4. Terminal demonstration coverage and limits.

The test suite establishes repeatable behavior for the stated cases. It is not proof that every possible input or environment is correct. Remaining limitations are described in Chapter 7 rather than represented as completed features.

6.6 Demonstration and team preparation

Step	Action	Explain during evaluation
1	Start MySQL and launch RoomWise.	External configuration, JDBC driver and package layers.
2	Search 2026-11-09 at 09:00; compare Occupied.	Weekly classes, dated bookings, half-open intervals.
3	Add a uniquely named DEMO room.	Validation, INSERT and generated primary key.
4	Add a booking, then refresh and edit its end time.	Create/read/update and object-to-row mapping.
5	Attempt a conflicting booking.	Guard-row lock, overlap query, rollback and error.
6	Delete demo booking, then delete the empty demo room.	Delete, referential integrity and safe cleanup.
7	Show ScheduledEvent and both subclasses.	Abstract behavior, inheritance and dynamic dispatch.
8	Run the test command and inspect its summary.	What was tested and what remains unverified.

Table 6.5. Suggested demonstration sequence.

6.7 Team readiness

Suggested presentation leads: Adithya Narayana Anand explains architecture, Java models and polymorphism; Aadithya T explains schema, JDBC and CRUD; Mitran Reddy explains verification, concurrency and the live demonstration. These are suggested presentation assignments, not claims about historical contribution. Each team member should rehearse the complete sequence and answer questions from all sections.

Before evaluation, every member should be able to explain why SQL values are bound, why equal booking boundaries are allowed, why a primary key does not prevent interval overlap, what rollback does, how the abstract method dispatch works, and why the current application is a trusted local prototype without authentication.

CHAPTER 7: SECURITY, LIMITATIONS, AND MAINTENANCE

7.1 Implemented security and integrity controls

All user data passed into SQL is bound using PreparedStatement. Dynamic SQL fragments select only fixed internal table/condition choices; user strings do not become SQL syntax. Validated constructors limit text lengths and reject invalid time ranges. Database keys, uniqueness, enum domains and CHECK constraints provide a second integrity layer. [2, 4]

Credentials are read from a local file outside the built JAR. The final report contains no database password. Users should keep this file private and grant the application only the necessary database privileges. Existing repository history may already include the local credential file from earlier work; ignoring a file does not remove historical copies. Changing repository history and rotating credentials require a separate coordinated maintenance action.

SQL errors are wrapped before being shown in the UI. The original cause remains available to debugging code. There is no centralized audit log, authentication, authorization, encrypted credential store or enforced remote TLS policy. These controls are not claimed as implemented.

7.2 Known limitations

Limitation	Practical implication
No login or role restrictions	Use only as a trusted local academic prototype.
Teacher identity is text	Aliases, homonyms and inconsistent spelling can produce incorrect identity matches.
Recurring schedule has no semester dates	A weekly entry applies to every matching weekday.
Same-day intervals only	Overnight events must be redesigned or split.
Source-data overlaps retained	Imported timetable rows can already overlap; they are not silently repaired.
Global guard serializes writes	Clear correctness mechanism, limited write concurrency.

Table 7.1. Security and scheduling limitations.

7.3 Performance and maintenance plan

Availability loads event lists through the DAO and checks them with ordinary Java loops. This makes the logic easy to explain, but large datasets may require targeted date/weekday SQL queries. Each terminal command waits for its database result. Future work should focus on measured query performance and output paging while retaining the same service contract.

Room deletion is deliberately restricted when dependent schedules exist. An operator must review and remove or reassign those records first. The application does not automatically archive deleted bookings, and there is no undo feature. Before substantial data maintenance, take a database backup and verify that restoration works.

Maintenance activity	Procedure / frequency
Before a demonstration	Check MySQL service, local configuration, Java version and report-matching build.
After a code change	Run model tests and relevant integration tests; update concept map and report if behavior changed.
Before a schema change	Back up data, define migration, test it on a separate database, preserve foreign keys.
After timetable updates	Review duplicate/overlapping source rows and verify representative room/date queries.
For credential changes	Update private config; keep built artifacts and repository history under review.
For data recovery	Restore backup to a separate database first; verify counts and sample relationships before replacing live data.

Table 7.2. Maintenance responsibilities.

7.4 Troubleshooting

If connection fails, verify that MySQL is running and db.properties is in the working directory or selected with roomwise.config. If a write is rejected, check the room ID, duplicate room name, text length, date/time format and overlapping room/teacher events. If --seed refuses to run, it is protecting existing data; do not delete records simply to bypass that check. If a generated JAR is locked, close the process using it before running Maven clean.

CHAPTER 8: CONCLUSION AND FUTURE WORK

8.1 Conclusion

RoomWise now provides a working integration of Java OOP and relational database operations. Its Java model distinguishes a dated booking from a recurring timetable entry through inheritance and overridden behavior. The service uses those objects to compute room availability, while the persistence interface and JDBC implementation keep database details separate from the UI.

Rooms, bookings and timetable entries each support Create, Read, Update and Delete. The implementation uses generated primary keys, room foreign keys, input validation, prepared parameters and explicit transactions. A database guard row prevents two cooperating application writers from both accepting a conflicting room or teacher schedule. Nineteen automated tests passed for the documented snapshot.

The report connects concepts to actual classes and operations instead of describing only theoretical features. It includes the current SQL schema, source-code images, recorded terminal output images, verification boundaries and a demonstration script. The prototype remains suitable for a controlled academic demonstration; deployment to a wider campus audience requires identity, authorization and operational improvements.

8.2 Prioritized future work

Priority	Enhancement	Reason
1	Authenticated accounts and authorized management roles	Protect schedule modification from untrusted users.
2	Stable faculty/course identities and normalized offerings	Resolve name matching and repeated source text.
3	Semester periods, holidays and exception dates	Represent a real academic calendar.
4	Targeted DAO queries, targeted date queries and paged terminal output	Improve responsiveness as the dataset grows.
5	Audit history, soft cancellation and restoration workflow	Support traceability and recovery.
6	Capacity/equipment filters and utilization reporting	Extend room discovery after core data quality is established.

Table 8.1. Future development priorities.

REFERENCES

[1] Oracle. Object-Oriented Programming Concepts. The Java Tutorials.
https://docs.oracle.com/javase/tutorial/java/concepts/
Accessed 9 November 2026.

[2] Oracle. Using Prepared Statements. JDBC Basics.
https://docs.oracle.com/javase/tutorial/jdbc/basics/prepared.html
Accessed 9 November 2026.

[3] Oracle. Using Transactions. JDBC Basics.
https://docs.oracle.com/javase/tutorial/jdbc/basics/transactions.html
Accessed 9 November 2026.

[4] Oracle / MySQL. FOREIGN KEY Constraints. MySQL 8.0 Reference Manual.
https://dev.mysql.com/doc/refman/8.0/en/create-table-foreign-keys.html
Accessed 9 November 2026.

[5] RoomWise project source. src/main/java/roomwise/, pom.xml and database/schema.sql. Local implementation snapshot verified 9 November 2026. Primary evidence for all implementation claims.

[6] Supplied campus timetable exports: database/data/theory.json and database/data/labs.json; generated SQL: database/real_data.sql. Data supplied with the project; no independent publication date was provided.

[7] ModelTest and DatabaseIntegrationTest. Local Maven Surefire results, 9 November 2026: 19 tests, zero failures and zero errors in the explicitly enabled complete run.

[8] NAMMA_CHENNAI.pdf, supplied by the user. Used only as a visual and structural reference for the cover, REC logo, preliminary pages and certificate alignment. Its project content, authorship, code and test results are not part of RoomWise.

The Java tutorial pages above describe foundational concepts and note their JDK 8 origin. The actual RoomWise project targets Java 17 and was verified with JDK 21. Source-specific implementation claims in this report are supported by the local code and test results, not inferred from the tutorial examples.

APPENDIX A: SQL SCHEMA

The following listing is the exact current database/schema.sql. It creates missing tables and the synchronization row without dropping existing application data. Main respects the configured database; manual execution of the file selects roomwise_db. The Java initializer performs the documented legacy foreign-key migration separately.

```
-- RoomWise: timetable and booking schema
-- Run through RoomWise automatically, or execute in MySQL Workbench.

CREATE DATABASE IF NOT EXISTS roomwise_db
    CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

USE roomwise_db;

CREATE TABLE IF NOT EXISTS rooms (
    id   INT         NOT NULL AUTO_INCREMENT PRIMARY KEY,
    room VARCHAR(40) NOT NULL UNIQUE
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS timetable_slots (
    id          INT          NOT NULL AUTO_INCREMENT PRIMARY KEY,
    room_id     INT          NOT NULL,
    day_of_week ENUM('MONDAY','TUESDAY','WEDNESDAY','THURSDAY',
                     'FRIDAY','SATURDAY','SUNDAY') NOT NULL,
    start_time  TIME         NOT NULL,
    end_time    TIME         NOT NULL,
    course_code VARCHAR(40)  NOT NULL,
    course_name VARCHAR(180) NOT NULL,
    section     VARCHAR(180) NOT NULL,
    teacher     VARCHAR(150) NOT NULL,
    class_type  ENUM('THEORY','LAB') NOT NULL,
    INDEX idx_slots_day_room_time(day_of_week, room_id, start_time, end_time),
    CONSTRAINT chk_slot_times CHECK (end_time > start_time),
    CONSTRAINT fk_slot_room_restrict FOREIGN KEY (room_id) REFERENCES rooms(id)
        ON DELETE RESTRICT
) ENGINE=InnoDB;
```

Appendix A / SQL schema continued

```
CREATE TABLE IF NOT EXISTS bookings (
    id           INT          NOT NULL AUTO_INCREMENT PRIMARY KEY,
    teacher      VARCHAR(150) NOT NULL,
    room_id      INT          NOT NULL,
    booking_date DATE         NOT NULL,
    start_time   TIME         NOT NULL,
    end_time     TIME         NOT NULL,
    INDEX idx_bookings_date_room_time(booking_date, room_id, start_time, end_time),
    CONSTRAINT chk_booking_times CHECK (end_time > start_time),
    CONSTRAINT fk_booking_room FOREIGN KEY (room_id) REFERENCES rooms(id)
        ON DELETE RESTRICT
) ENGINE=InnoDB;


-- Internal mutex for atomic room and teacher schedule checks.
CREATE TABLE IF NOT EXISTS schedule_guard (
    id TINYINT NOT NULL PRIMARY KEY,
    CONSTRAINT chk_single_guard CHECK (id = 1)
) ENGINE=InnoDB;
INSERT IGNORE INTO schedule_guard(id) VALUES (1);
```

A.1 Initial data and migration notes

Use the --seed command only for first-time population of an empty schema. It checks all three domain tables under the guard lock and imports the supplied SQL inside a transaction. Normal startup does not compare the timetable to a fixed row count and does not reload the source data. This preserves legitimate CRUD changes.

For an existing legacy schema, the initializer replaces the timetable-room CASCADE foreign key with a RESTRICT relationship. This prevents a room deletion from silently removing imported class rows. The migration is tested with retained room and timetable fixtures.

A.2 Useful read-only demonstration queries

```
SELECT COUNT(*) AS rooms FROM rooms;
SELECT COUNT(*) AS timetable_rows FROM timetable_slots;
SELECT id, teacher, booking_date, start_time, end_time
FROM bookings ORDER BY id;
SELECT r.room, b.booking_date, b.start_time, b.end_time
FROM rooms r JOIN bookings b ON b.room_id = r.id;
```

APPENDIX B: OUTPUT SCREENSHOTS

This appendix contains images rendered from real terminal output captured on 9 November 2026. The same ConsoleMenu class used by the application read recorded input, called CampusService and JdbcCampusDao, and printed these results. Images are labeled transcript renderings; they are not fabricated terminal-window captures. Repeated menus are omitted from some images, and long result lists are explicitly marked as excerpts.

A temporary database was initialized from the supplied 157 rooms and 4,167 timetable rows. The room and booking demonstrations then created room 158 (DEMO-SEMINAR), booking 1 and timetable slot 4168. All demo records were deleted through the menu, and the temporary database was removed after recording. The configured campus database was not used for demo mutations.

Evidence	Observed result
Point search before demo insertion	2026-11-09 09:00: 91 free rooms.
Room create/update	Room 158 created as DEMO-ROOM and renamed DEMO-SEMINAR.
Booking create/read/update	Booking 1: 18:00-19:00 changed to 18:00-19:30; persisted row read back.
Conflict	18:30-19:00 booking rejected for the already occupied demo room.
Timetable create/update/read	Slot 4168 changed from 17:00-18:00 to 17:00-17:30; occupied result reflects change.
Delete and cleanup	Demo booking, slot and room removed; final isolated counts 157 / 4,167 / 0.

Table B.1. Terminal evidence provenance.

The supplied input sequences, complete output logs and TerminalEvidence.java recorder are included in the archived report sources. The images preserve output values; captions explain the selected commands and input fields. The screen-oriented report from the previous build is superseded by this terminal-only edition.

Terminal menu

```
ROOMWISE | Captured terminal output | menu


========== ROOMWISE ==========
1. Find free rooms at a time
2. View occupied events
3. Manage rooms
4. Manage bookings
5. Manage timetable
6. Find free rooms for an interval
0. Exit
Choose: Goodbye.
```

Figure B.1. Terminal menu (captured-output rendering).

Launch RoomWise and select 0 to exit. The menu exposes six operations plus Exit.

Evidence source: menu.txt; input sequence: menu-inputs.txt. Full logs are retained; repeated menu blocks may be omitted in this image.

Free-room search output

```
ROOMWISE | Captured terminal output | free
```

```
Choose: Date (YYYY-MM-DD): Time (HH:MM):
FREE ROOMS | 2026-11-09 | 09:00
3     A104/105-A              Free until 10:00
4     A104/105-B              Free until 10:00
13    A208/209-B              Free until 13:20
14    A210/211                Free until 23:59:59
15    A210/211-A              Free until 11:40
16    A210/211-B              Free until 11:40
33    B First Floor           Free until 23:59:59
34    B Ground Floor          Free until 23:59:59
35    B Second Floor          Free until 23:59:59
40    B125                    Free until 23:59:59
41    B126                    Free until 23:59:59
42    B128                    Free until 23:59:59
43    B205                    Free until 23:59:59
44    B206                    Free until 23:59:59
[... middle room rows omitted from this excerpt ...]
Total free rooms: 91
Choose: Goodbye.
```

Figure B.2. Free-room search output (captured-output rendering).

Inputs: 1; date 2026-11-09; time 09:00; 0. The result contains 91 rooms. The image shows the beginning and final count of the actual output.

Evidence source: free.txt; input sequence: free-inputs.txt. Full logs are retained; repeated menu blocks may be omitted in this image.

Create and update a room

```
ROOMWISE | Captured terminal output | room-crud

Choose: C Create / R Read / U Update / D Delete: Room name: Created room #158
Choose: C Create / R Read / U Update / D Delete: Room ID: New room name: Updated
 room #158
Choose: Goodbye.
```

Figure B.3. Create and update a room (captured-output rendering).

Inputs: Manage rooms / C / DEMO-ROOM, then U / 158 / DEMO-SEMINAR. MySQL assigns the new ID; the update preserves that ID.

Evidence source: room-crud.txt; input sequence: room-crud-inputs.txt. Full logs are retained; repeated menu blocks may be omitted in this image.

Create and read a booking

```
ROOMWISE | Captured terminal output | booking-create

Choose: C Create / R Read / U Update / D Delete: Room ID: Teacher: Date (YYYY-
MM-DD): Start (HH:MM): End (HH:MM): Created booking #1
Choose: C Create / R Read / U Update / D Delete: #1 | room 158 | 2026-11-09 |
18:00-19:00 | Demo Faculty
Total bookings: 1
Choose: Goodbye.
```

Figure B.4. Create and read a booking (captured-output rendering).

Inputs: Manage bookings / C; room 158; Demo Faculty; 2026-11-09; 18:00; 19:00. R reads back the generated booking ID and stored fields.

Evidence source: booking-create.txt; input sequence: booking-create-inputs.txt. Full logs are retained; repeated menu blocks may be omitted in this image.

Update and read a booking

```
ROOMWISE | Captured terminal output | booking-update

Choose: C Create / R Read / U Update / D Delete: Booking ID: Room ID: Teacher:
Date (YYYY-MM-DD): Start (HH:MM): End (HH:MM): Booking updated.
Choose: C Create / R Read / U Update / D Delete: #1 | room 158 | 2026-11-09 |
18:00-19:30 | Demo Faculty
Total bookings: 1
Choose: Goodbye.
```

Figure B.5. Update and read a booking (captured-output rendering).

Inputs: Manage bookings / U; booking 1; same room, teacher and date; 18:00 to 19:30. R confirms the persisted new end time.

Evidence source: booking-update.txt; input sequence: booking-update-inputs.txt. Full logs are retained; repeated menu blocks may be omitted in this image.

Reject an overlapping booking

```
ROOMWISE | Captured terminal output | conflict

Choose: C Create / R Read / U Update / D Delete: Room ID: Teacher: Date (YYYY-
MM-DD): Start (HH:MM): End (HH:MM): Action failed: Room or teacher overlaps an
existing booking.
Choose: Goodbye.
```

Figure B.6. Reject an overlapping booking (captured-output rendering).

Inputs: new booking for room 158, Other Teacher, 2026-11-09, 18:30-19:00. The existing 18:00-19:30 reservation causes rejection; no second booking commits.

Evidence source: conflict.txt; input sequence: conflict-inputs.txt. Full logs are retained; repeated menu blocks may be omitted in this image.

Polymorphic occupied-event output

```
ROOMWISE | Captured terminal output | occupied

Choose: Date (YYYY-MM-DD): Time (HH:MM):
OCCUPIED EVENTS | 2026-11-09 | 18:15
Room 158 | 18:00-19:30 | Booking | Demo Faculty
Active events: 1 | Distinct rooms: 1
Choose: Goodbye.
```

Figure B.7. Polymorphic occupied-event output (captured-output rendering).

Inputs: 2; 2026-11-09; 18:15. The Booking object is returned as ScheduledEvent and prints its overridden description.

Evidence source: occupied.txt; input sequence: occupied-inputs.txt. Full logs are retained; repeated menu blocks may be omitted in this image.

Timetable create, update and read

```
ROOMWISE | Captured terminal output | timetable

Choose: C Create / R Read / U Update / D Delete: Room ID: Teacher: Weekday:
Start (HH:MM): End (HH:MM): Course code: Course name: Section: THEORY or LAB:
Created timetable slot #4168
Choose: C Create / R Read / U Update / D Delete: Timetable slot ID: Room ID:
Teacher: Weekday: Start (HH:MM): End (HH:MM): Course code: Course name: Section:
 THEORY or LAB: Timetable slot updated.
Choose: Date (YYYY-MM-DD): Time (HH:MM):
OCCUPIED EVENTS | 2026-11-09 | 17:15
Room 158 | 17:00-17:30 | THEORY | DEMO-OOP Java Demonstration | Demo Faculty
Active events: 1 | Distinct rooms: 1
Choose: Goodbye.
```

Figure B.8. Timetable create, update and read (captured-output rendering).

Create a MONDAY demo class for room 158 at 17:00-18:00, update its end to 17:30, then query 2026-11-09 at 17:15. The result shows the persisted updated class.

Evidence source: timetable.txt; input sequence: timetable-inputs.txt. Full logs are retained; repeated menu blocks may be omitted in this image.

Delete records and verify cleanup

```
ROOMWISE | Captured terminal output | delete

Choose: C Create / R Read / U Update / D Delete: Booking ID: Type DELETE to
confirm: Deleted booking #1
Choose: C Create / R Read / U Update / D Delete: Total bookings: 0
Choose: C Create / R Read / U Update / D Delete: Timetable slot ID: Type DELETE
to confirm: Deleted timetable slot #4168
Choose: C Create / R Read / U Update / D Delete: Room ID: Type DELETE to
confirm: Deleted room #158
Choose: Goodbye.
```

Figure B.9. Delete records and verify cleanup (captured-output rendering).

Delete booking 1 and read zero bookings; delete slot 4168; then delete unused room 158. Each deletion requires the exact confirmation DELETE.

Evidence source: delete.txt; input sequence: delete-inputs.txt. Full logs are retained; repeated menu blocks may be omitted in this image.

Appendix B / Final verification record

Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.013 s -- in roomwise.ModelTest

Tests run: 15, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.650 s -- in roomwise.DatabaseIntegrationTest

All recorded console sessions used actual ConsoleMenu, CampusService and JDBC.
Final isolated data: 157 rooms / 4167 timetable slots / 0 bookings.

The current build contains 16 production Java source files in the required package layers. The source diagrams and code images correspond to this terminal-only implementation. Signatures and the practical-examination date remain blank for the team and institution to complete.